

MADONNA ARIAS AGUDELO - 60929

04/10/2026

<https://github.com/madonnaarias60929-cmyk/explorador-interactivo-2026-2>

Instrucciones

Duración: 75 minutos. Individual. Temas de las semanas 1 a 5. Vale el 20% de la nota del curso. Sin IA: Copilot, Cursor, ChatGPT y similares deshabilitados. Celular guardado. No se permite consultar material. Parte A (50%) se responde en esta hoja. Parte B (50%) se hace en el computador. Sugerencia: 30 min para la A y 45 para la B.

En las preguntas que piden explicar, se califica el porqué, no solo el resultado.

Parte A · Teórico-práctica (50%)

1. HTTP y la web 10% Un usuario reporta: «la página carga, pero las imágenes salen rotas». En DevTools → pestaña Network, las peticiones de las imágenes muestran el estado 404 .

(a) ¿Qué significa exactamente ese 404 ? ¿El problema está del lado del cliente o del servidor? Justifica.

El 404 significa que el navegador sí logró conectarse al servidor, pero que el archivo o imagen que estaba pidiendo no se encontró en esa dirección. El problema es principalmente del **cliente** (en este caso, de nosotros al programar), porque probablemente escribimos mal la ruta o el nombre del archivo en el código HTML.

(b) ¿Qué dato de la petición fallida revisarías primero para encontrar la causa, y con qué lo compararías?

Revisaría la **URL solicitada (Request URL)** de la imagen en la pestaña Network de las DevTools. Después, la compararía con la ruta real y el nombre exacto del archivo dentro de las carpetas de mi proyecto para ver en qué le erré al escribirlo.

2. HTML semántico 10% Reescribe este fragmento con las etiquetas semánticas correctas y justifica dos de tus decisiones.

```
<div class="top">
  <div class="logo">Mi blog</div>
</div>
<div class="links">...</div>
<div class="post">
  <div class="titulo-grande">Mi
  viaje</div>
```

```
<p>...</p>
</div>
```

Justificaciones:

```
<header class="top">

<h1 class="logo">Mi blog</h1>

</header>

<nav class="links">

  <!-- Enlaces de navegación -->

</nav>

<main>

  <article class="post">

    <h2>Mi viaje</h2>

    <p>...</p>

  </article>

</main>
```

Justificaciones:

1. Usé `<header>` para la parte de arriba porque es la cabecera y metí el menú dentro de un `<nav>` para indicar claramente cuales son los enlaces de navegación de la página.
2. Usé `<article>` para la entrada del blog porque es un contenido completo que se entiende por sí solo, acompañado de un `<h2>` para mantener la jerarquía correcta de los títulos.

3. Especificidad: ¿qué regla gana? 10%

```

        color: purple; }
p { color: black; } #zona p { color: green; }
.destacado { color: orange; } p.destacado {
        <div id="zona">
        <p class="destacado">Texto</p> </div>

```

(a) ¿De qué color se ve «Texto»? Explica por qué, calculando la especificidad de las reglas que aplican.

Se ve de color **verde**. Gana la regla **#zona p** porque tiene un selector de ID (su especificidad es **1, 0, 1**), y los IDs mandan sobre las clases o las etiquetas solas.

(b) Si se elimina la regla **#zona p**, ¿de qué color queda y por qué?

Queda de color **púrpura**. Al quitar la del ID, quedan compitiendo **.destacado (0, 1, 0)** y **p.destacado (0, 1, 1)**. Como **p.destacado** combina una etiqueta y una clase, gana porque su especificidad es un poquito mayor

4. Box model 10% Una caja tiene **width: 300px; padding: 24px; border: 2px solid; margin: 16px;**

(a) Calcula el ancho visible de la caja (borde a borde, sin contar el margin) con **box-sizing: content-box** y con **box-sizing: border-box**. Muestra la operación.

content-box: $300px(\text{ancho}) + 48px(\text{paddingizqyder}) + 4px(\text{bordeizqyder}) = 352px$.

border-box: Se queda en **300px** cerrados, porque con este modo el padding y los bordes ya vienen incluidos dentro de los 300px que le pusimos.

(b) ¿Por qué el curso pone ***, ::before, ::after { box-sizing: border-box; }** como primera línea de toda hoja de estilos?

Lo ponemos para no enloquecernos sumando los paddings y los bordes a mano. Así, si le decimos a una caja que mida 300px, medirá exactamente eso y no se nos va a desajustar ni a romper el diseño de la página.

Modo	Operación	Ancho visible
content-box	$300 + (24 \times 2) + (2 \times 2) = 300 + 48 + 4$	352px
border-box	300 (el padding y borde ya vienen incluidos)	300px

5. Flexbox, Grid y responsive 10% Una grilla de productos debe mostrar 1 columna en el celular y hasta 4 en

escritorio. Dos estudiantes proponen:

```
/* Propuesta A */
.grilla { display: flex; flex-wrap: wrap; gap: 16px; }
.tarjeta { flex: 1 1 250px; }
/* Propuesta B */
```

```
.grilla {
  display: grid; gap: 16px;
  grid-template-columns:
    repeat(auto-fit, minmax(250px, 1fr)); }
```

(a) ¿En qué situación visible se comportan distinto las dos propuestas? Descríbela (puedes dibujarla).

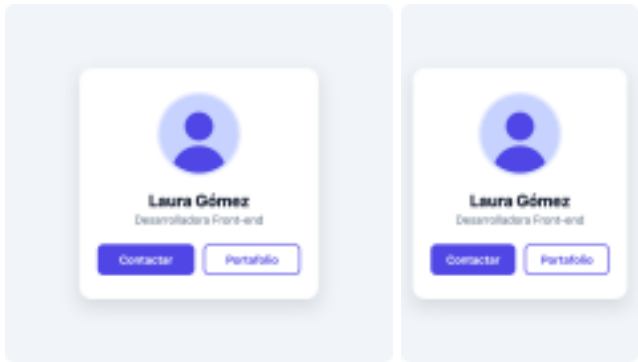
Se notan diferente en la última fila si sobran espacios. Con **Flexbox (Propuesta A)** las tarjetas de la última línea se estiran a lo ancho para rellenar el espacio vacío; en cambio con **Grid (Propuesta B)** conservan su tamaño mínimo y no se estiran por la fuerza, dejando espacio libre a la derecha si no se completan las columnas.

(b) ¿Cuál usarías para una grilla uniforme de tarjetas y por qué?

Usaría la **Propuesta B (Grid)** porque es mucho más limpia para hacer grillas de este tipo. Maneja sola las filas y columnas y evita que las tarjetas se deformen o se estiren de más en la última línea.

Parte B · Ejercicio práctico (50%)

Maqueta la tarjeta de perfil de la imagen. Puedes usar CSS puro o Tailwind por CDN, a tu elección. Las imágenes de referencia y el archivo [avatar.svg](#) están en el aula virtual (carpeta recursos-estudiante).



Vista en escritorio: la tarjeta queda centrada en la pantalla. Vista a 320px: ocupa el ancho disponible sin desbordarse.

Medidas de referencia

Elemento	Especificación
Fondo de la página	Gris claro #f1f5f9 . La tarjeta, centrada vertical y horizontalmente en el viewport.
Tarjeta	Fondo blanco, ancho máximo 320px , relleno 32px arriba/abajo y 24px a los lados, esquinas de 16px , sombra suave, texto centrado.
Foto	Circular, 112px × 112px , borde de 4px color #e0e7ff , 16px de separación con el nombre. Usa avatar.svg .

Nombre	«Laura Gómez», 22px , negrita, color #0f172a .
Cargo	«Desarrolladora Front-end», 15px , color #64748b , 24px de separación con los botones.
Botones	Lado a lado, mismo ancho, 12px entre ellos, esquinas de 8px , texto en negrita. Contactar: fondo #4f46e5 , texto blanco. Portafolio: fondo blanco, borde de 2px y texto #4f46e5 .

Requisitos mínimos 7

1. HTML semántico: la tarjeta es un `<article>` , el nombre un encabezado (`<h2>`) y los botones son `<button>` reales.
2. La foto tiene un texto `alt` apropiado.
3. La tarjeta queda centrada vertical y horizontalmente en la pantalla (con Flexbox o Grid).
4. Los dos botones tienen un estado `:hover` visible (cambio de color de fondo, por ejemplo).
5. No se rompe ni se desborda entre 320px y 1200px de ancho. Compruébalo con el modo responsive de DevTools.

Entrega

En tu repositorio de talleres, crea la carpeta `parcial-1/` con `index.html` (y `estilos.css` si usas CSS puro) más `avatar.svg` . Haz commit y push antes de la hora escrita en el tablero, con el mensaje `Parcial 1 - <tu nombre>` . No se califican commits posteriores a esa hora. Si tu repositorio de talleres está dividido por semanas o no tienes uno, crea un repositorio nuevo llamado `web1-parcial-1` . En cualquier caso, escribe la URL en el encabezado de esta hoja.